

Función asincrónica



Línea base Ampliamente disponible



La declaración **de función asíncrona** crea una [asignación](#) de una nueva función asíncrona a un nombre propio. La palabra clave está permitida dentro del cuerpo de la función, lo que permite que el comportamiento asíncrono basado en promesas se escriba de forma más limpia y evita la necesidad de configurar explícitamente cadenas de promesas. `await`

También puedes definir funciones asíncronas usando la [expresión de funciones asíncronas](#).

Pruébalo

Demo de JavaScript: declaración de función asíncrona

```
1 function resolveAfter2Seconds() {  
2   return new Promise((resolve) => {  
3     setTimeout(() => {  
4       resolve("resolved");  
5     }, 2000);  
6   });  
7 }  
8  
9 async function asyncCall() {  
10  console.log("calling");  
11  const result = await resolveAfter2Seconds();  
12  console.log(result);  
13  // Expected output: "resolved"  
14 }  
15  
16 asyncCall();  
17
```

Sintaxis

JS

```
async function name(param0) {  
  statements  
}  
  
async function name(param0, param1) {  
  statements  
}  
  
async function name(param0, param1, /* ..., */ paramN) {  
  statements  
}
```

 **Nota:** No puede haber un terminador de línea entre y , de lo contrario se [inserta](#) automáticamente un punto y coma, lo que hace que se convierta en un identificador y el resto en una declaración. `async function async function`

Parámetros

name

El nombre de la función.

Param

Opcional

El nombre de un parámetro formal para la función. Para la sintaxis de los parámetros, véase la [referencia Functions](#).

Sentencias

Opcionales

Las afirmaciones que componen el cuerpo de la función. Se puede usar el mecanismo. `await`

Descripción

Una declaración crea un objeto `AsyncFunction`. Cada vez que se llama a una función asíncrona, devuelve una nueva [Promesa](#) que se resolverá con el valor devuelto por la función asíncrona, o se rechazará con una excepción no capturada dentro de la función asíncrona. `async function`

Las funciones asíncronas pueden contener cero o más `expresiones de espera`. Las expresiones de espera hacen que las funciones que devuelven promesas se comporten como si fueran sincrónicas suspendiendo la ejecución hasta que la promesa devuelta se cumpla o sea rechazada. El valor resuelto de la promesa se trata como el valor de retorno de la expresión `await`. El uso de `yield` permite el uso de bloques ordinarios `/` alrededor de código asíncronico. `async` `await` `try` `catch`

❗ **Nota:** La palabra clave solo es válida dentro de funciones asíncronas dentro del código JavaScript normal. Si lo usas fuera del cuerpo de una función asíncronica, obtendrás un `SyntaxError`. `await`

`await` puede usarse por sí sola con `módulos JavaScript`.

❗ **Nota:** El propósito de `/` es simplificar la sintaxis necesaria para consumir APIs basadas en promesas. El comportamiento de `/` es similar a combinar `generadores` y promesas. `async` `await` `async` `await`

Las funciones asíncronas siempre devuelven una promesa. Si el valor de retorno de una función asíncrona no es explícitamente una promesa, estará implícitamente envuelta en una promesa.

Por ejemplo, consideremos el siguiente código:

JS

```
async function foo() {  
  return 1;  
}
```

Es similar a:

JS

```
function foo() {  
  return Promise.resolve(1);  
}
```

Ten en cuenta que, aunque el valor de retorno de una función asíncrona se comporta como si estuviera envuelta en un `Promise`, no son equivalentes. Una función asíncrona devolverá una *referencia* diferente, mientras que devuelve la misma referencia si el valor dado es una promesa. Puede ser un problema cuando quieres comprobar la igualdad entre una promesa y el valor de retorno de una función asíncrona. `Promise.resolve` `Promise.resolve`

JS

```
const p = new Promise((res, rej) => {
  res(1);
});

async function asyncReturn() {
  return p;
}

function basicReturn() {
  return Promise.resolve(p);
}

console.log(p === basicReturn()); // true
console.log(p === asyncReturn()); // false
```

El cuerpo de una función asíncrona puede considerarse dividido por cero o más esperas expresiones. Código de nivel superior, hasta e incluyendo la primera expresión `await` (si existe uno), se ejecuta de forma sincrónica. De este modo, una función asíncrona sin expresión `await` funcionará de forma sincrónica. Si existe una expresión de espera dentro del cuerpo de la función, sin embargo, la función asíncrona siempre se completará de forma asíncrona.

Por ejemplo:

JS

```
async function foo() {
  await 1;
}
```

```
}
```

También es equivalente a:

JS

```
function foo() {  
  return Promise.resolve(1).then(() => undefined);  
}
```

El código después de cada expresión de espera puede considerarse como existente en una callback. De este modo, se construye progresivamente una cadena de promesas con cada reentrante Pasa por la función. El valor de retorno forma el eslabón final de la cadena. `.then`

En el siguiente ejemplo, esperamos sucesivamente dos promesas. El progreso avanza funcionan en tres etapas. `foo`

1. La primera línea del cuerpo de funciones se ejecuta de forma sincrónica, con la expresión de espera configurada con la promesa pendiente. El progreso a través se suspende entonces y el control se devuelve a la función que llamado. `foo foo foo`
2. Algún tiempo después, cuando la primera promesa se haya cumplido o rechazado, Control vuelve a entrar en `.` El resultado del primer cumplimiento de la promesa (si no fue rechazada) se devuelve de la expresión de espera. Aquí está asignado a `.` El progreso continúa, y los segundos esperan expresión se evalúa. De nuevo, el progreso se suspende y el control lo está cedió. `foo 1 result1 foo`
3. Algún tiempo después, cuando la segunda promesa se haya cumplido o rechazado, Control vuelve a entrar en `.` El resultado de la segunda resolución de promesa es `Regresó` de la segunda expresión de espera. Aquí se asigna a `.` Control se traslada a la expresión de retorno (si la hay). El predeterminado el valor de retorno de se devuelve como el valor de resolución de la Promesa actual. `foo 2 result2 undefined`

JS

```
async function foo() {  
  const result1 = await new Promise((resolve) =>  
    setTimeout(() => resolve("1")),  
  );  
  const result2 = await new Promise((resolve) =>  
    setTimeout(() => resolve("2")),  
  );  
}
```

```
);  
}  
foo();
```

Fíjate en cómo la cadena de promesas no se construye de una sola vez. En cambio, la cadena de promesas es construido en etapas a medida que el control se cede sucesivamente desde y se devuelve al asíncrono funcional. Por ello, debemos ser conscientes del comportamiento de manejo de errores al tratar con operaciones asíncronas concurrentes.

Por ejemplo, en el siguiente código se lanzará un error de rechazo de promesa no gestionado, incluso si un manejador ha sido configurado más adelante en la promesa cadena. Esto se debe a que no estará "cableado" en la cadena de promesas hasta que control retorna desde `.catch p2 p1`

JS

```
async function foo() {  
  const p1 = new Promise((resolve) => setTimeout(() => resolve("1"), 1000));  
  const p2 = new Promise( (_, reject) =>  
    setTimeout(() => reject(new Error("failed")), 500),  
  );  
  const results = [await p1, await p2]; // Do not do this! Use Promise.all or Promise.allSettled instead.  
}  
foo().catch(() => {}); // Attempt to swallow all errors...
```

`async function` Las declaraciones se comportan de forma similar a las `declaraciones de función`: se `elevan` a la parte superior de su ámbito y pueden llamarse en cualquier parte de su ámbito, y solo pueden redeclararse en ciertos contextos.

Ejemplos

Funciones asíncronas y orden de ejecución

JS

```
function resolveAfter2Seconds() {
  console.log("starting slow promise");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("slow");
      console.log("slow promise is done");
    }, 2000);
  });
}

function resolveAfter1Second() {
  console.log("starting fast promise");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("fast");
      console.log("fast promise is done");
    }, 1000);
  });
}

async function sequentialStart() {
  console.log("== sequentialStart starts ==");

  // 1. Start a timer, log after it's done
  const slow = resolveAfter2Seconds();
  console.log(await slow);

  // 2. Start the next timer after waiting for the previous one
  const fast = resolveAfter1Second();
  console.log(await fast);

  console.log("== sequentialStart done ==");
}
```



```
async function sequentialWait() {
  console.log("== sequentialWait starts ==");

  // 1. Start two timers without waiting for each other
  const slow = resolveAfter2Seconds();
  const fast = resolveAfter1Second();

  // 2. Wait for the slow timer to complete, and then log the result
  console.log(await slow);
  // 3. Wait for the fast timer to complete, and then log the result
  console.log(await fast);

  console.log("== sequentialWait done ==");
}
```

```
async function concurrent1() {
  console.log("== concurrent1 starts ==");

  // 1. Start two timers concurrently and wait for both to complete
  const results = await Promise.all([
    resolveAfter2Seconds(),
    resolveAfter1Second(),
  ]);
  // 2. Log the results together
  console.log(results[0]);
  console.log(results[1]);

  console.log("== concurrent1 done ==");
}
```

```
async function concurrent2() {
  console.log("== concurrent2 starts ==");
```

```
// 1. Start two timers concurrently, log immediately after each one is done
await Promise.all([
  (async () => console.log(await resolveAfter2Seconds()))(),
  (async () => console.log(await resolveAfter1Second()))(),
]);
console.log("== concurrent2 done ==");
}

sequentialStart(); // after 2 seconds, logs "slow", then after 1 more second, "fast"

// wait above to finish
setTimeout(sequentialWait, 4000); // after 2 seconds, logs "slow" and then "fast"

// wait again
setTimeout(concurrent1, 7000); // same as sequentialWait

// wait again
setTimeout(concurrent2, 10000); // after 1 second, logs "fast", then after 1 more second, "slow"
```

Espera y concurrencia

En , la ejecución suspende 2 segundos para el primero , y luego otro segundo para el segundo . El El segundo temporizador no se crea hasta que el primero ya se ha disparado, por lo que el código termina Después de 3 segundos. `sequentialStart` `await` `await`

En , ambos temporizadores se crean y luego se ed. Los temporizadores se ejecutan simultáneamente, lo que significa que el código termina en 2 segundos en lugar de 3, es decir, el temporizador más lento. Sin embargo, las llamadas siguen en serie, lo que significa que la segunda espera a que termine la primera. En este caso, el resultado de El temporizador más rápido se procesa después del más lento. `sequentialWait` `await` `await` `await`

Si deseas realizar otros trabajos de forma segura después de que dos o más trabajos se hayan completado simultáneamente, debes esperar una llamada a `Promise.all()` o `Promise.allSettled()` antes de ese trabajo.

 **Advertencia:** Las funciones y no son funcionalmente equivalentes. `sequentialWait` `concurrent1`

En , si la promesa rechaza antes de que se cumpla, entonces un error de rechazo de promesa no manejado será elevado, independientemente de si el llamante ha configurado una cláusula de captura. `sequentialWait` `fast` `slow`

En , se ajusta la promesa cadena de una sola vez, lo que significa que la operación fallará rápido independientemente del orden de el rechazo de las promesas, y el error siempre ocurrirá dentro de la configuración cadena de promesas, permitiendo que se atrapara de la manera normal. `concurrent1 Promise.all`

Reescribir una cadena de promesas con una función asincrónica

Una API que devuelva una `Promesa` dará lugar a una cadena de promesas, y divide la función en muchas partes. Consideremos el siguiente código:

JS

```
function getProcessedData(url) {  
  return downloadData(url) // returns a promise  
    .catch((e) => downloadFallbackData(url)) // returns a promise  
    .then((v) => processDataInWorker(v)); // returns a promise  
}
```

Puede reescribirse con una única función asincrónica de la siguiente manera:

JS

```
async function getProcessedData(url) {  
  let v;  
  try {  
    v = await downloadData(url);  
  } catch (e) {  
    v = await downloadFallbackData(url);  
  }  
  return processDataInWorker(v);  
}
```

Alternativamente, puedes encadenar la promesa con: `catch()`

JS

```
async function getProcessedData(url) {
  const v = await downloadData(url).catch((e) => downloadFallbackData(url));
  return processDataInWorker(v);
}
```

En las dos versiones reescritas, observa que no hay una sentencia después de la palabra clave, aunque eso también sería válido: El valor de retorno de un función asíncrona está implícitamente envuelta en `Promise.resolve` - si No es ya una promesa en sí misma (como en los ejemplos). `await` `return`

Especificaciones

Especificaciones
ECMAScript® 2027 Especificación del Lenguaje# sec-async-function-definitions

Compatibilidad con navegadores

[Reporta problemas con estos datos de compatibilidad](#) • [Ver datos en GitHub](#)

	Chrome	Arista	Firefox	Ópera	Safari	Chrome Android	Firefox para Android	Ópera Android	Safari en iOS	Samsung Internet	WebView Android	WebView en iOS	Bollo	Deno	Node.js
<code>async function</code> Enunciado	✓ 55	✓ 15	✓ 52	✓ 42	✓ 10.1	✓ 55	✓ 52	✓ 42	✓ 10.3	✓ 6	✓ 55	✓ 10.3	✓ 1	✓ 1	✓ 7.6

Véase también

- [Guía de funciones](#)
- [Guía de Uso de Promesas](#)
- [Funciones](#)
- `AsyncFunction`
- [Expresión de la función asincrónica](#)
- `function`
- `function*`
- `async function*`
- `await`
- `Promise`
- [Decorando funciones JavaScript asíncronas](#) [↗](#) en innolitics.com (2016)



Tu plan para un internet mejor.